시스템 분석 및 설계

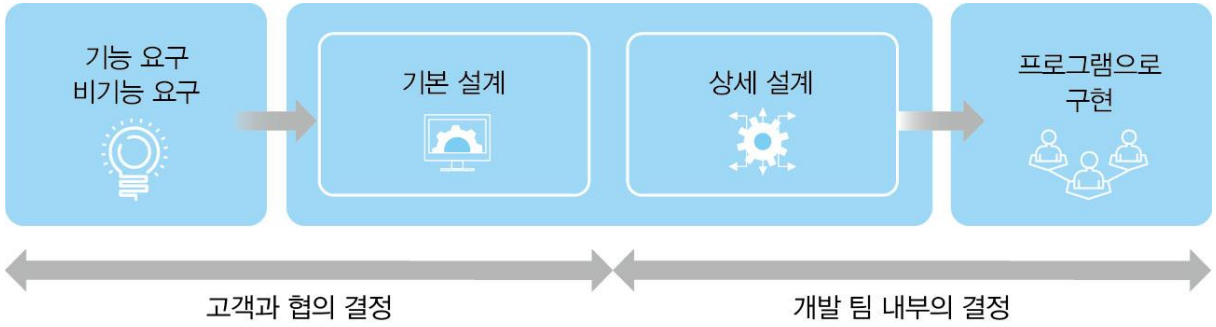
07주차. 상세 설계(1)

서울사이버대학교

학습목표

1. 상세 설계의 목적과 과정을 설명할 수 있다.
2. 모듈 분할의 원리와 이를 위한 3개 기법을 이해한다.
3. 처리 로직 설계를 위한 다양한 기법과 도구를 탐색해 본다.

- 이전 장에서의 기본 설계는 요구사항의 <정의 결과> 를 <어떻게> 로 바꾸는 작업인데 반해, 상세설계는 어떻게 구현할지를 상세하게 개발자에게 전달하기 위한 것
- 상세설계는 대략적으로 설계한 기본 설계의 결과를 상세하게 설계하는 것이 아니라, 분명한 목적이 있고, 기본 설계와는 다른 관점으로 작업해야 함



설계 과정의 개요

상세 설계는 기본 설계의 내용을 충족하면서 품질 높은 원시 코드를 생성하기 위한 설계서

- 기본 설계서를 바탕으로 프로그램의 내부 데이터를 결정하고, 로직 등 시스템의 구체적인 구현 방법을 정의
    - 예를 들어 구조적 설계는 톱다운 방식으로 기능을 분할하여 모듈을 정의하며, 또한 객체지향 프로그램의 경우 클래스의 조합과 기능을 결정하고, 서브시스템을 모듈로 분할함
- 기본 설계의 명세를 충족하도록 모듈 안의 동작을 정의

"기본 설계서를 기반으로 원시 코드를 작성하는 것은 불가능한가?"

→ 꼭 그렇지는 않지만 프로그래밍에 더욱 가까운 설계서인 상세 설계서를 작성하는 목적은 명확함

① 프로그램 작성 시 오류를 방지하고 개발 초기 단계에서 품질을 높이기 위해

② 프로그래머의 능력에 관계 없이 프로그램의 품질을 균일하게 만들기 위해

③ 여러 개발 업체에서 하나의 시스템을 개발하는 경우 프로그램의 통합을 손쉽게 하기 위해

④ 프로그램의 재사용을 촉진하는 컴포넌트로 만들기 쉽게 하기 위해

개발 초기부터 품질을 잘 확보하려면 잘 정의된 상세한 설계서를 마련해야 함. 또한 시스템의 품질 향상을 위해서는 능력에 상관 없이 좋은 원시 코드를 작성할 수 있어야 함. 따라서 프로그래머가 잘 정의된 설계서대로 프로그래밍을 할 수 있도록 상세 설계가 필요하며, 이렇게 하면 품질이 일정한 프로그램으로 완성될 가능성이 높아짐

상세 설계의 범위는 시스템 내부에 집중되는 반면, 기본 설계는 시스템 외부에 대한 디자인이라고도 볼 수 있음

즉 상세설계는 외부로부터 보이지 않는 시스템의 내부이며, 기본 설계는 보이는 시스템의 바깥 부분을 대상으로 함

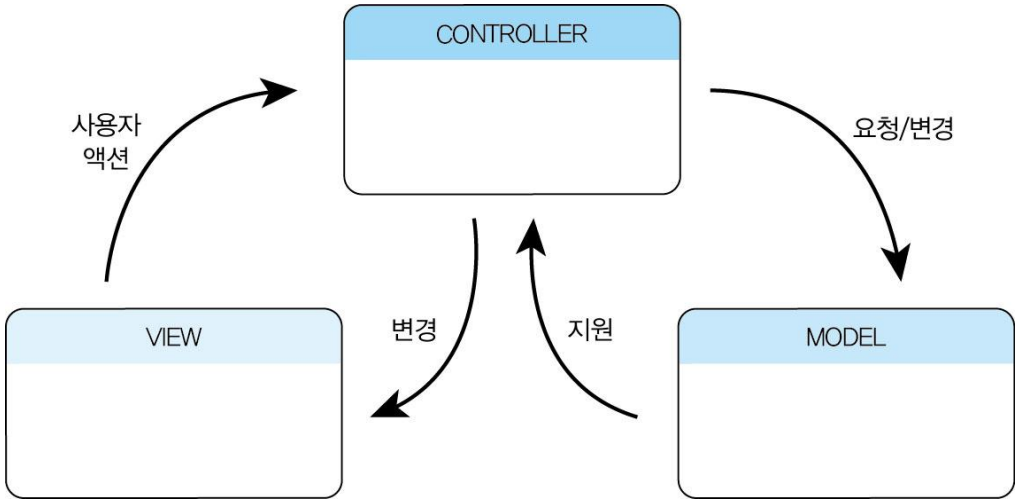
상세 설계, 기본 설계 각각 동일한 설계 과정 안에 두 가지 설계가 존재하는 이유는 이렇게 역할이 전혀 다르기 때문임

상세 설계는 기본 설계에서 정해진 기능을 담당하는 시스템 구성요소의 내부를 만들기 위한 것이고, 기본 설계는 고객이 사용할 실제 눈에 보이는 기능을 구축하기 위한 작업이라고도 볼 수 있음

상세 설계의 범위를 이해하기 위해서는 소프트웨어 구성 요소의 역할을 구별할 수 있어야 하며, 특히 설계 범위를 구별하는 데는 MVC 모델이 사용됨

MVC 모델은 사용자 인터페이스를 가진 시스템을 구축할 때 프로그램의 내부를 구성하는 디자인 패턴으로, 역할마다 분할하여 모듈을 나누어 프로그래밍 할 때 적합한 방법이며, 이 모델을 적용하여 시스템을 분할하면 기본 설계와 상세 설계의 범위를 확인할 수 있음

- 모델 (Model): 데이터를 접근하여 조작하는 비즈니스 로직이 되는 부분으로 주로 **상세 설계의 대상**이 됨
- 뷰 (View): 화면 표시와 입출력이 되는 부분으로 **기본 설계의 대상**이 됨
- 콘트롤러 (Controller): 모델과 뷰를 제어하는 부분으로 **상세 설계의 대상**이 됨



(예) 학생 성적 관리 시스템을 MVC로 구현

- **모델(Model):**
  - Student 클래스: 학생 정보(이름, 학번, 성적 등) 저장
  - GradeCalculator 클래스: 평균, 학점 계산 로직 처리
- **뷰(View):**
  - StudentListView: 학생 목록 화면 표시
  - GradeReportView: 성적표 화면 표시
  - InputFormView: 성적 입력 양식 표시
- **컨트롤러(Controller):**
  - StudentController: 사용자가 학생 정보를 추가/수정/삭제할 때 처리
  - GradeController: 성적 계산 요청 처리, 결과를 뷰에 전달

동작 흐름

- 사용자가 뷰(예: InputFormView)를 통해 학생 성적을 입력
- 컨트롤러(StudentController)가 사용자 입력을 받아 처리
- 컨트롤러가 모델(Student, GradeCalculator)에 데이터 업데이트 요청
- 모델이 데이터를 처리하고 비즈니스 로직 실행
- 컨트롤러가 업데이트된 정보를 가져와 뷰에 전달
- 뷰(GradeReportView)가 사용자에게 결과 표시

온라인 쇼핑몰:

- 모델: 상품, 장바구니, 사용자 정보 관리
- 뷰: 상품 목록, 상세 정보, 장바구니, 결제 화면
- 컨트롤러: 상품 검색, 장바구니 추가/삭제, 결제 처리

블로그:

- 모델: 게시글, 댓글, 사용자 정보 관리
- 뷰: 메인 페이지, 게시글 상세 페이지, 댓글 목록
- 컨트롤러: 게시글 작성/수정/삭제, 댓글 추가/삭제

음악 플레이어:

- 모델: 음악 파일 정보, 재생 목록 관리
- 뷰: 음악 목록, 재생 컨트롤 (재생/정지/다음 곡 등)
- 컨트롤러: 재생 제어, 재생 목록 관리

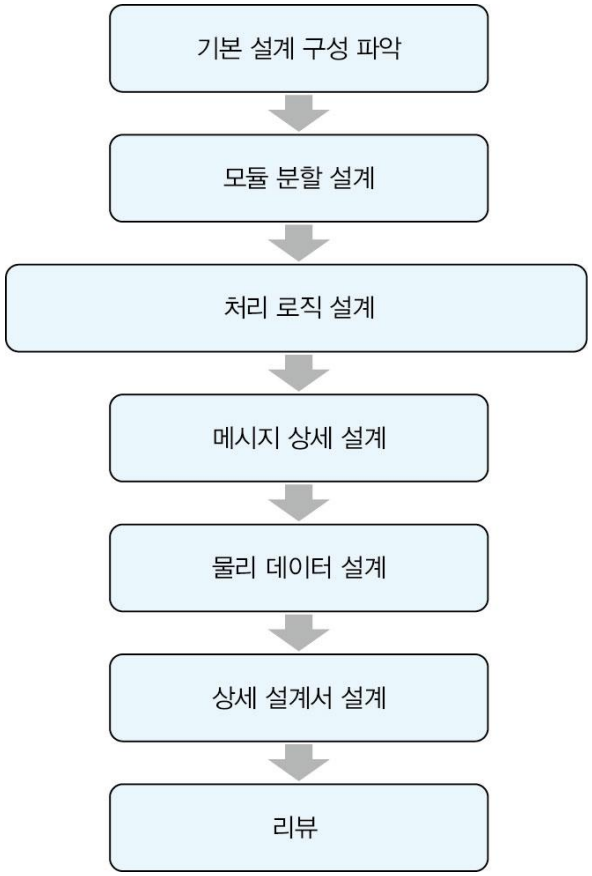
간단한 계산기:

- 모델: 계산 로직 수행
- 뷰: 숫자/연산자 버튼, 결과 화면
- 컨트롤러: 사용자 입력 처리, 계산 수행, 결과 표시

상세 설계의 구체적인 절차는 다음과 같음

상세 설계에서도 프로그래밍에 필요한 개별 디자인을 실시한 후 상세 설계서를 작성함. 상세 설계 개발 작업의 종류는 대상 시스템의 종류에 따라 다르지만, 비즈니스 시스템은 옆의 그림에 표시된 순서대로 작성하는 것이 일반적

- **기본 설계 구성 파악**: 시스템의 기본 요구사항과 구조를 이해하고 전체적인 설계 방향을 정하는 단계. 개발하려는 시스템의 목적과 핵심 기능을 파악
- **모듈 분할 설계**: 전체 시스템을 독립적인 기능 단위인 모듈로 분할. 각 모듈은 특정 기능을 담당하며, 이를 통해 개발의 복잡성을 줄이고 유지보수성을 향상
- **처리 로직 설계**: 각 모듈 내에서 데이터가 어떻게 처리될지 구체적인 알고리즘과 로직을 설계하는 단계. 비즈니스 로직과 데이터 처리 방식을 정의
- **메시지 상세 설계**: 모듈 간 통신 방식과 데이터 교환을 위한 메시지 형식, 프로토콜을 설계하는 단계. 시스템 컴포넌트 간 인터페이스를 정의
- **물리 데이터 설계:** 실제 데이터베이스 구조, 테이블, 관계 등을 정의. 논리적 데이터 모델을 실제 구현 가능한 물리적 스키마로 변환
- **상세 설계서 설계**: 이전 단계에서 결정된 모든 설계 내용을 문서화. 개발자들이 실제 구현 시 참조할 수 있는 상세한 설계 문서를 작성
- **리뷰:** 완성된 설계 문서를 팀원들과 함께 검토하는 단계. 설계상의 오류나 개선점을 찾아내고 수정하여 최종 설계를 확정



상세 설계 절차

소프트웨어는 프로그래밍 언어로 작성된 컴퓨터 명령어의 집합이지만, 일반적으로 다양한 기능과 처리 과정이 복잡하게 얽혀 있음. 이러한 크고 복잡한 소프트웨어를 기능과 데이터 등을 작은 단위로 분할하여 독립적으로 처리할 수 있는 구조로 만들고, 이 분할된 요소들을 조합하여 원하는 처리를 구현하는 방식을 모듈화(또는 모듈 분할)라고 함

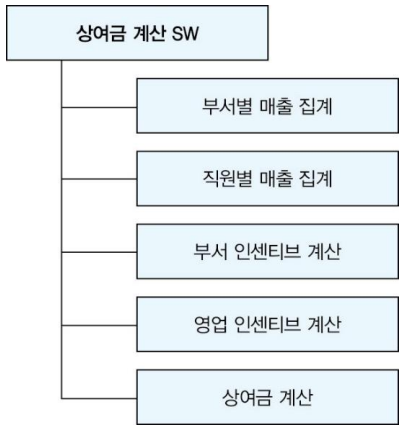
예) 영업 사원에 대해 인센티브를 계산하는 소프트웨어

[경험이 적은 개발자]

- 회계 기간 말 부서별 인센티브 계산
- 해당 금액을 영업 사원의 목표 매출 달성도를 배분
- 상여금에 반영(계산)

[경험이 많은 개발자]

아래와 같이 추후 변경과 관리를 위해 소단위로 분할

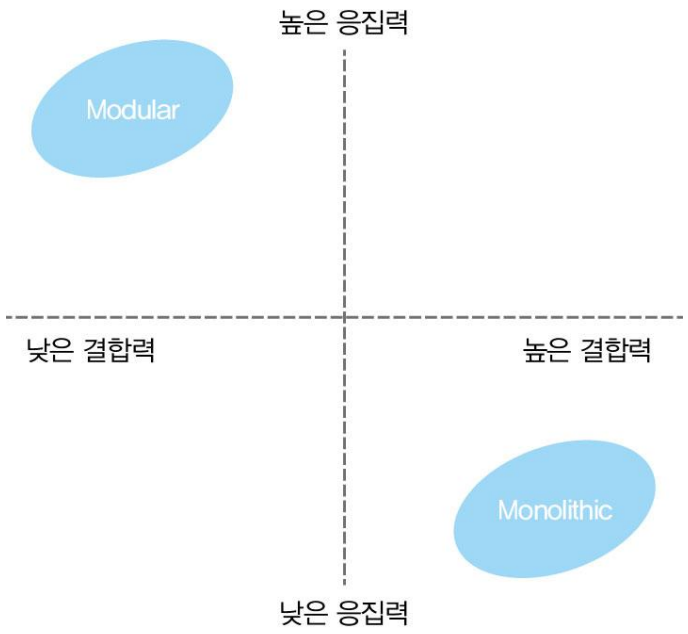


모듈을 어떤 크기로 분할하면 좋은가? 에 대한 답을 주는 것이 모듈 분할 원리

- 작게 분할하면 개별 모듈은 간단하고 범용성이 높아짐
- 그러나 인터페이스가 복잡해 지고 모듈 관리가 복잡해지고 모듈 관리가 힘들어짐

모듈 분할은 2가지 요소(모듈 안의 요소들이 얼마나 관련되어 있는지를 뜻하는 응집력과, 모듈 밖 다른 모듈과의 관계가 얼마나 결합되어 있는지를 말하는 결합도)를 고려해야 함

- 높은 응집력: 모듈이 하는 일을 측정. 여러가지 문제를 해결하는 대신 매우 응집력 있는 모듈은 단일 목적이나 책임에 중점을 둠. 이것을 객체지향 설계에서는 단일 책임의 원칙이라고 함
- 결합력: 서로 다른 모듈이 서로 얼마나 의존하는지 측정한 것. 모듈을 가능한 한 낮게 결합되어야 함. 어떤 모듈이 다른 모듈의 호출이 많아 결합이 높다면 의존성이 높아져 다른 모듈의 작업에 종속적인 모듈이 되기 때문



모듈형(modular)**과 덩어리형**(monolithic)

모듈형 애플리케이션은 응집력이 높고 결합력이 낮은 모듈로 구성된 애플리케이션

- 이 원리를 따르지 않고 많은 모듈을 만들 수 있지만 이런 경우 응용 프로그램은 덩어리형 시스템이 됨
- 이런 경우 시스템의 유지보수가 어려워짐

모듈형 애플리케이션으로 구성하면 개발팀이 애플리케이션을 기능별 수직 슬라이스로 구축하고, 점진적으로 펼쳐 나가면서 쉽게 연결하고 추가하여 이동시킬 수 있음, 즉 새로운 기능을 개발할 때에도 쉽게 확장하고 연결할 수 있음

이러한 모듈 분할에는 주로 3가지 기법이 사용됨

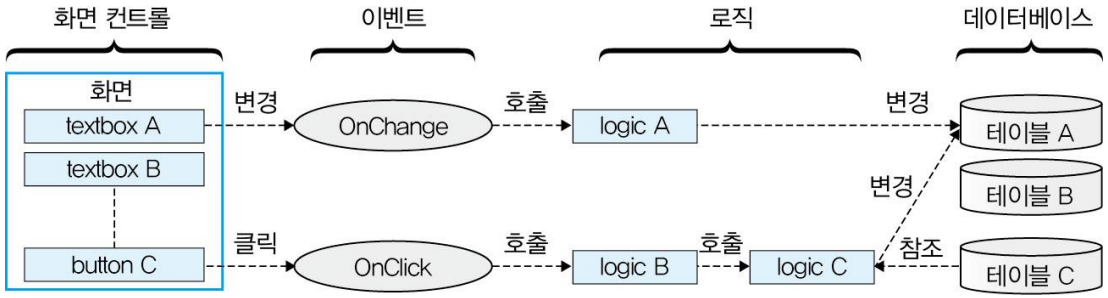
- TR 분할법: 이벤트나 트랜잭션(입력 데이터)에 착안하여 분할하는 방법
- STS 분할법: 데이터의 흐름에 주목하여 분할하는 방법
- 객체지향 설계: 자료와 처리의 집합(클래스)의 관점으로 정보 은닉 원리를 따라 분할하는 방법

TR 분할법((Transaction Refinement or Transaction Partitioning)은 이벤트나 트랜잭션(입력 데이터) 에 착안하여 시스템을 분할하는 방법

- 시스템이 처리해야 할 다양한 트랜잭션을 중심으로 기능을 구성하는 설계 방식으로, 주로 정보 시스템이나 트랜잭션 기반 애플리케이션에서 많이 활용
- 시스템이 수행하는 주요 트랜잭션(입력 이벤트)을 분석하여 이를 기준으로 기능을 나누는 방식
- 하나의 트랜잭션이 처리될 때 수행되는 일련의 과정들을 기준으로 모듈을 정의하며 다양한 입력(이벤트)이 발생할 수 있으며, 입력 유형에 따라 다른 처리가 수행

아래 그림은 이벤트 중심의 모듈 구성을 표현. 화면 A에 배치된 텍스트 박스와 버튼 등의 컨트롤에서 클릭 이벤트가 발생하거나 값을 바꾸면 onChange or onClick 이벤트가 발생하며, 이를 처리하는 트랜잭션 중심으로 모듈을 바꿈

기본설계에서는 화면 컨트롤과 이벤트를 정의하였다면, 상세 설계에서는 처리 내용을 모듈로 분할하며, 우측의 처리 로직을 정의하는데 집중

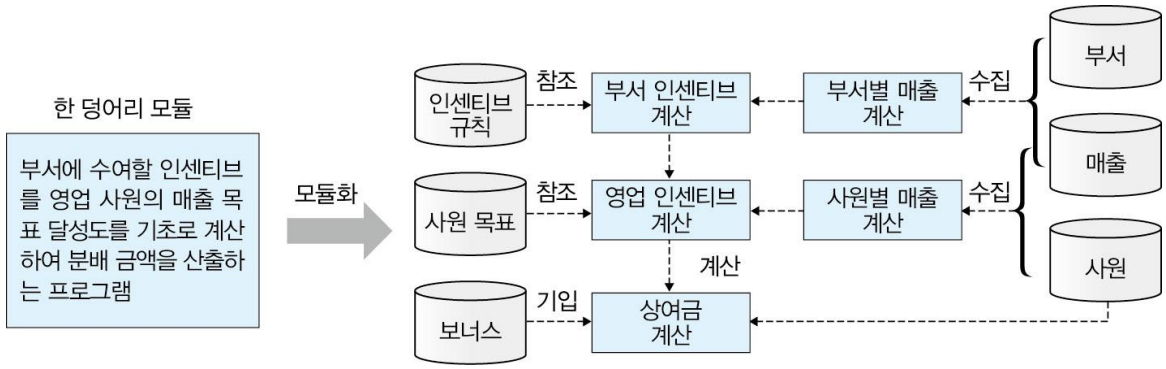


이벤트 중심 모듈 분할

STS 분할법 (Source-Transform-Sink Partitioning)은 데이터의 흐름에 주목하여 시스템을 분할하는 방법

- 시스템이 처리해야 할 데이터가 입력(Source)에서 시작하여 변환(Transform)을 거쳐 출력(Sink)으로 흘러가는 구조를 분석하여 시스템을 모듈화하는 방식
- 데이터의 흐름을 기준으로 시스템을 설계 (입력(Source) → 변환(Transform) → 출력(Sink) 구조)
- 데이터가 어떤 방식으로 변형되고 최종적으로 어떤 결과를 만들어내는지 분석하여 분할
- 어디서 호출되든 입력과 출력 인터페이스를 명확히 파악하면 호출에 의존하지 않고 모듈을 파악할 수 있음

아래 그림은 5개의 모듈로 분할하였으나 매출 수집, 인센티브 계산을 하나로 통합하여 3개로 분할할 수도 있음. 하지만 하나의 모듈로 하거나 인센티브 계산과 상여금 계산 2개의 모듈로만 분할할 경우 모듈 내부의 응집이 낮게 되어 바람직한 모듈이라고 보기는 어려움



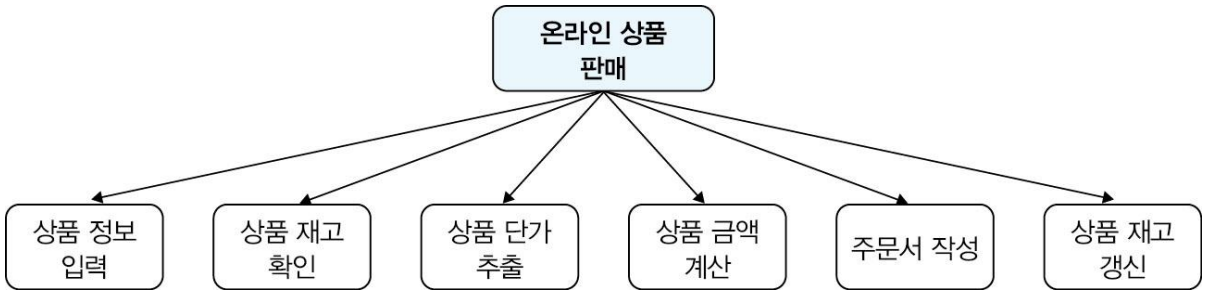
STS **분할법**

구조적 설계는 요구 분석 업무에서 업무 흐름을 파악하여 사용하는 DFD를 기초로 하여 소프트웨어 모듈 구조와 모듈 간 호출 관계, 모듈의 실행 순서 등을 정의하는 방법이며, 모듈을 분할하여 구조도를 사용

- 구조도를 사용하면 개발대상 소프트웨어의 구조를 계층적으로 정의할 수 있음
- 구조도의 결과를 이용하여 모듈의 소스코드를 작성하고 이를 조합하면 소프트웨어 전체를 만들 수 있음

아래 그림에서 표현된 구조도는 온라인 상품 판매 업무를 표현

- 상품 정보 입력 → 상품 재고 확인 → 상품 단가 추출 → 대금을 계산하여 주문서 작성 → 상품 재고를 갱신.. 순서로 모듈이 실행됨을 직관적으로 보여줌

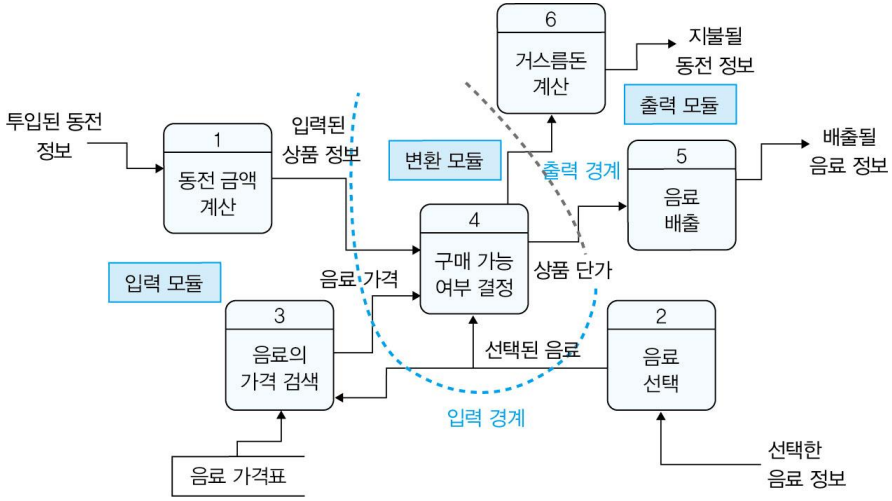


구조도

구조적 설계에서는 DFD를 분석하여 프로세스 중에서 [데이터 입력], 데이터를 이용하여 작업을 수행하는 [처리], 처리 결과를 [출력]하는 기능을 식별하여 구조도를 작성함

구조도를 작성하는 절차는 다음과 같음

1. **부모 모듈 식별**
   - DFD에서 각 프로세스의 실행을 제어하는 기능을 찾아 최상위 부모 모듈로 만듦
   - 각 기능의 실행을 제어하는 프로세스가 없다면 실행을 관리하는 상위 모듈을 새로 생성
   - 이 때 DFD 전체 프로세스를 요약할 내용을 부모 모듈의 이름으로 정의
2. **모듈의 실행 순서 결정**
   - DFD 중에서 입력, 변환, 출력을 찾아 이를 바탕으로 각 기능의 호출 순서를 결정함
   - 중요한 기능을 찾아내는 변환 분석을 통하여 부모 모듈을 발견하고 모듈 구조를 결정



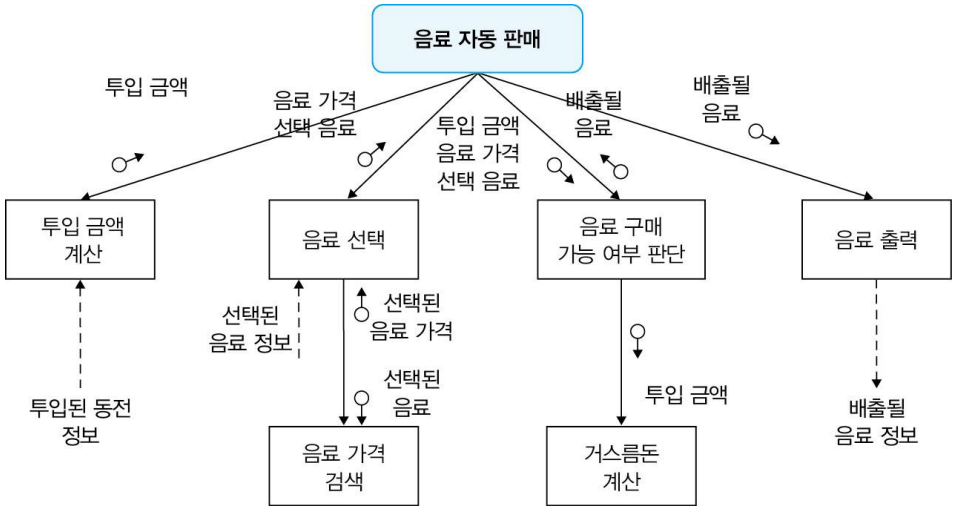
음료 판매기 DFD 변환 분석

위 사례를 대상으로 모듈을 분할하는 과정은 다음과 같음

- 입력 모듈
  - 투입 금액을 계산하는 프로세스는 동전 금액 데이터를 입력받기 때문에 입력 부분
  - 음료를 선택하는 프로세스는 음료에 대한 데이터를 입력하는 과정이기 때문에 입력 부분
  - 음료의 가격을 검색하는 프로세스는 음료 선택 프로세스와 연결(=호출)되기 때문에 입력 부분
- 변환 모듈
  - 음료 구매 가능 여부 판단 프로세스는 음료에 대한 정보를 받아서 구매 여부를 판정하기 때문에 변환 부분
  - 거스름돈을 계산하는 프로세스는 음료 구매 가능 여부 판정 프로세스에서 호출되어 출력되기 때문에 변환 부분
- 출력
  - 음료를 배출하는 프로세스는 음료에 대한 데이터를 출력하고 있기 때문에 출력 부분
- 작성된 DFD 내부에는 프로세스의 실행을 제어하는 프로세스가 없기 때문에 부모 모듈이 될 음료 자판기라는 모듈을 추가

이와 같은 분석을 거치면 모듈의 호출 구조는 아래와 같이 파악

- 호출 관계가 표시되어 있음
- DFD에 표현된 데이터 흐름을 참고하여 모듈에서 주고받는 데이터들을 표현



음료 자동판매기의 구조도

시스템을 작은 모듈로 분할하는 작업이 끝나면 상세 설계에서 해야 할 중요한 작업 중 하나는 모듈 안의 처리 로직을 설계하는 일

- 이 단계에서 정의하는 것은 구조도나 UML 클래스 다이어그램에서 도출된 가장 작은 단위의 업무 프로세스
- 모듈의 처리 로직은 독립적인 목적을 완수해야 할 최소 단위의 업무 프로세스라고 할 수 있음

프로세스 단위 업무 흐름을 분석하고 자세히 정의하는 일은 소프트웨어 설계에서 가장 상세하고 정확해야 함

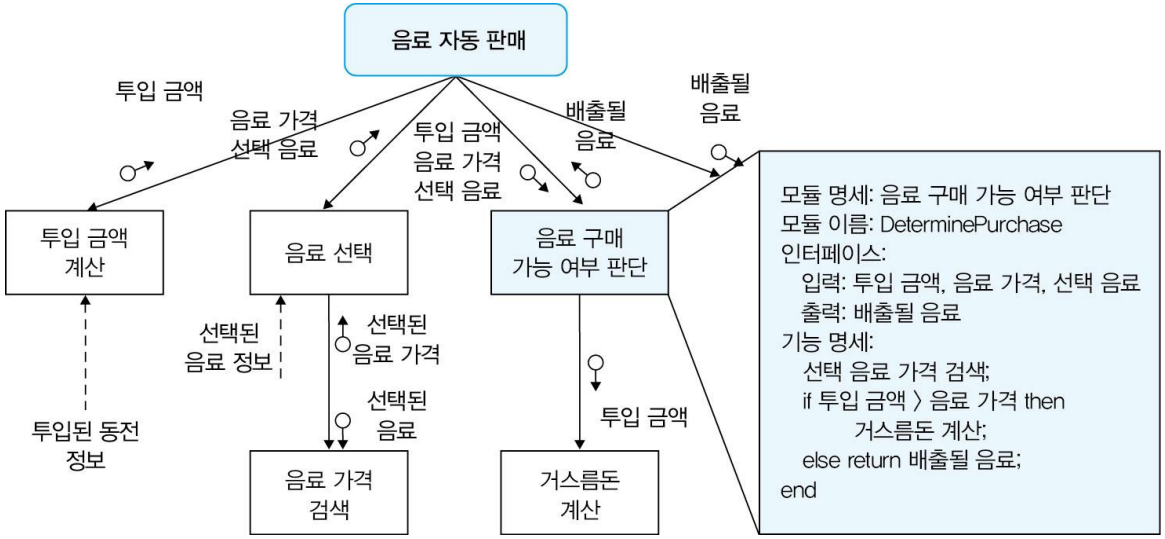
- 가장 작은 단위의 업무는 메뉴에서 항목을 선택하고 어떤 대상을 조작하는 화면을 열고, 작업 수행을 종료하고, 화면을 닫을 때 까지의 일련의 행동을 의미
- 업무 프로세스의 최소 단위는 대개 <어떤 기능이 무엇을 한다> 라는 하나의 독립적인 목적을 달성하는 덩어리
- 프로세스 로직은 구체적인 소프트웨어 동작 이미지가 들어있는 일련의 동작 단계라고 생각하면 좋음

프로세스 또는 모듈의 상세한 로직을 나타내는 대표적 기법

- N-S 차트
- 모듈 명세서
- IPO 차트

모듈 기능 설명 (예)

- **입력 정보:** 투입 금액 정보, 음료 가격 정보, 선택 음료 정보
- **출력 정보:** 배출될 음료
- **판단 로직:**
  - 선택 음료 가격을 검사
  - 만약 투입 금액이 음료 가격보다 크거나 같으면 (투입 금액 > 음료 가격 then): 거스름돈을 계산
  - 그렇지 않으면(else): 배출될 음료를 반환합니다 (return 배출될 음료)
  - 종료(end)

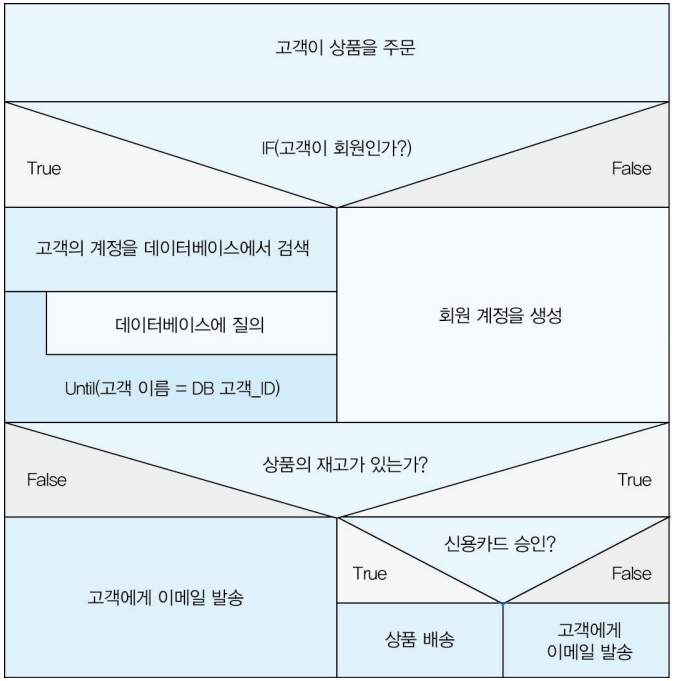


처리 로직 설계의 예

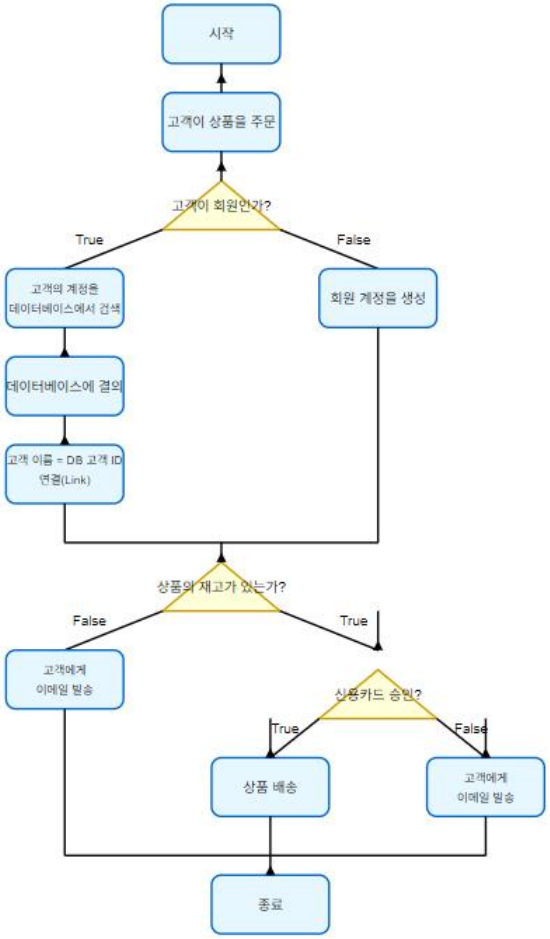
나시-슈나이더만 차트(Nassi-Shneiderman Chart): 구조적 플로우차트(Structogram), 나시슈나이더만도 라고도 불리며, 1972년 아이작 나시와 벤 슈나이더만이 개발한 프로그래밍 설계 도구로, 알고리즘의 논리적 흐름을 시각화하여 구조적 프로그래밍을 지원

- 프로세스의 알고리즘을 주로 나타내는 방법으로 알고리즘 설계에 대해 그림으로 나타내기 때문에 흐름도만큼 다이어그램의 크기가 크지 않은 것이 최대 장점
- 흐름도를 박스 형태로 그린 것이라고도 할 수 있음
- 흐름도의 세 가지 제어(순차, 선택, 반복)에 의한 처리를 보여주며, 위에서 아래로 읽어내려감
- 가장 큰 특징은 로직이 흐름도보다 더 구조화된 형태로 표현된다는 점, 즉 로직이 하나의 시작점과 종료점을 갖게 됨
- 그러나 프로세스가 처리하는 데이터에 대한 설명은 다소 부족한 편

현재는 UML과 같은 현대적 설계 도구들이 산업에서 더 널리 사용되고 있지만, 나시-슈나이더만 차트는 여전히 컴퓨터 과학 교육 및 알고리즘 설계의 기초를 이해하는 데 중요한 도구



N-S 차트의 예



흐름도로 변환한 예

학습정리

- 상세 설계는 구체적인 구현 방법을 개발자(팀)에게 전달하는 것을 전제로 하는 작업으로 고품질의 원시코드를 생성하기 위한 설계서이다.
- MVC 모델은 상세 설계 및 기본 설계의 범위를 구별하여 파악할 수 있는 유용한 기법이다.
- 소프트웨어를 기능과 데이터 등을 작은 단위로 분할하여 독립적으로 처리할 수 있는 구조로 만들고, 이 분할된 요소들을 조합하여 원하는 처리를 구현하는 방식을 모듈화라고 하며, 높은 응집력과 낮은 결합력을 갖도록 설계해야 한다.
- 이렇게 설계한 모듈은 독립적이며 완전한 형태의 최소 단위의 업무 프로세스에 대응하며, 이를 구체화 하기 위한 처리 로직 설계가 반드시 이행되어야 한다.